Supplement: Syllabus Topics the Lectures Did Not Cover

ES 335 / ES 678 Machine Learning, Sem I 2026 — Prof. Udit Bhatia Sources: `course-documents/ml2026_syllabus_calendar_policies_grading.pdf` (lecture calendar, L1–L22), `course-documents/ml2026_course_roadmap_thoughtmap.pdf` · Previous: XGBoost and comparison

Why this file exists. The syllabus promises these topics up to the trees/ensembles lecture, but no lecture PDF covers them: **leakage and distribution shift** (syllabus L8), **validation design, baselines and ablations** (L12, L17), **optimization: learning rates, convergence, SGD** (L9–L10), **regression diagnostics** (L12), **multiclass classification** (L14–L15), **trees II: regression splits, categorical features, missing values, feature importance** (L19), and **KNN and the curse of dimensionality** (L21, scheduled before SVMs). They appear in the course objectives, so they can be examined. Everything here reuses the objects of L01–L12; the worked examples use the **twelve runs** of L12a where possible.

What this file is really about:

$$\text{honest evaluation (A1–A4)} \rightarrow \text{reliable fitting (A5–A6)} \rightarrow \text{more than two classes (A7)} \rightarrow \text{trees ir}$$

Step-up path

You need. Train and test error, and at least one fitted model.

1. Ask what the test set actually promises.
2. List the ways test information leaks into training.
3. Ask whether the deployment world matches the training world.
4. Choose a validation design that matches how the data arose.

You can stop when you can read an evaluation and name its most likely flaw.

Contents

Part A — Maths from the ground up

1. Generalization: what the test set promises
2. Data leakage: four ways the test set gets contaminated
3. Distribution shift: when the future is not the past

4. Validation design: CV variants, baselines, ablations, learning curves
5. Optimization: learning rate, convergence, SGD
6. Regression in practice: R-squared and residual diagnostics
7. Multiclass classification: softmax, one-vs-rest, macro/micro
8. Trees II: regression splits, categories, missing values, importance
9. K-nearest neighbours
10. The curse of dimensionality

Part C — Practice

Part A Maths From the Ground Up

A1 Generalization: what the test set promises

Step 1 — The quantity we care about is the risk $R(f) = \mathbb{E}_{(x,y)\sim P}[\ell(y, f(x))]$ (L02, slide 28). We can only compute the empirical risk $\hat{R}_{\text{train}}(f)$.

Step 2 — The gap. Because $\hat{f}$ was chosen to make $\hat{R}_{\text{train}}$ small, $\hat{R}_{\text{train}}(\hat{f})$ is **biased low**. The **generalization gap** is $R(\hat{f}) - \hat{R}_{\text{train}}(\hat{f})$. A deep tree (L12a A9) has training error 0 and a large gap.

Step 3 — What a test set gives you. If the test rows are (i) drawn from the **same** P as future data, (ii) **independent** of training, and (iii) **never used to make a choice**, then $\hat{R}_{\text{test}}(\hat{f})$ is an unbiased estimate of $R(\hat{f})$. Each of the next three sections breaks one of these conditions:

condition	broken by	section
(ii) independent of training	leakage	A2
(i) same P as the future	distribution shift	A3
(iii) not used to choose	tuning on test, reusing a validation set too often	A4

A2 Data leakage: four ways the test set gets contaminated

Definition. Leakage = information that **would not be available at prediction time** reaches the model during training or evaluation. The test score then looks better than deployment will be.

kind	example	fix
Target leakage	predicting "patient has pneumonia" with the feature "took antibiotics" (recorded *after* diagnosis)	keep only features known **before** the prediction moment
Preprocessing leakage	standardising (or imputing, or selecting features) using the mean/sd of **all** rows, then splitting	fit every preprocessing step **inside** the training fold (L08 slide 51); use a pipeline
Temporal leakage	random split of time-ordered data: the model trains on Friday and is tested on Thursday	split by time: train on the past, test on the future
Group/duplicate leakage	the same patient (or a near-duplicate image) in both train and test	split by group (A4)

Worked check — preprocessing leakage with feature selection. Take 50 rows of **pure noise** with 10 000 random features and random labels. Choose the 20 features most correlated with y on **all 50 rows**, then cross-validate a classifier on those 20. CV accuracy comes out far above 50%, although no real signal exists: the selection already looked at the test folds. Done correctly (selection inside each fold), CV accuracy is ≈ 50%. This is the classic example in ESL §7.10.2 (`Readings/4-Elements-of-statistical-learning.pdf`).

Smell test. A score that is *too* good (AUC 0.99 on a hard problem), or one feature with a huge importance (A8) that makes no causal sense, usually means leakage. Compare the shortcut paradox, L01 slides 18–19: the model can be right for the wrong reason.

A3 Distribution shift: when the future is not the past

Write the joint as $P(x, y) = P(x)P(y \mid x) = P(y)P(x \mid y)$. Each kind of shift changes one factor:

shift	what changes	what stays	example
Covariate shift	$P(x)$	$P(y \mid x)$	model trained on daytime photos, used at night
Prior (label) shift	$P(y)$	$P(x \mid y)$	disease prevalence 10% in the hospital, 2% in screening
Concept drift	$P(y \mid x)$	—	what counts as spam changes over the years

Worked example — prior shift breaks precision but not recall. A classifier has recall (TPR) 0.8 and FPR 0.1 (L10 A3). By Bayes' rule

$$\text{precision} = \frac{\text{TPR} \cdot \pi}{\text{TPR} \cdot \pi + \text{FPR} \cdot (1 - \pi)}.$$

- Hospital, $\pi = 0.10$: $\frac{0.08}{0.08+0.09} = 0.471$.
- Screening, $\pi = 0.02$: $\frac{0.016}{0.016+0.098} = 0.140$.

The same model, unchanged TPR and FPR (so the **ROC curve and AUC are unchanged**), now raises about six false alarms for each true case instead of about one. The cost-optimal threshold (L09 slide 24) also moves, because it depends on the probability, and $P(y = 1 \mid x)$ changes with π.

What to do. Make the validation data look like deployment (A4: time splits, group splits); monitor input statistics and the rate of positive predictions after deployment; recalibrate (L09 slide 22) when the base rate changes.

A4 Validation design: CV variants, baselines, ablations, learning curves

Step 1 — Recap. Hold-out and K-fold CV were introduced in L07 A11 and L08 A7. The variants below exist because the rows are not always iid.

variant	how the folds are made	use when
K-fold	random split into K parts	iid rows, enough data (K = 5 or 10)
Stratified K-fold	each fold keeps the class proportions	classification, especially rare positives (a random fold may have **no** positives)
Group K-fold	all rows of one group (patient, user, site) go to the same fold	repeated measurements (A2, group leakage)
Time-series split	train on folds $1..k$, validate on fold $k+1$ (expanding window)	time-ordered data (A2, temporal leakage)
Leave-one-out (LOOCV)	$K = n$	very small n; nearly unbiased, high variance, costly
Nested CV	inner CV chooses hyperparameters, outer CV scores the **whole** procedure	you need an honest score **and** you tune

Step 2 — Why nested CV. If you try 100 values of λ and report the best CV score, that score is the **maximum of 100 noisy estimates**, so it is biased upward (condition (iii) of A1). The outer loop scores "tune-then-fit" as a single procedure on data it never saw.

Step 3 — The one-standard-error rule. CV scores are noisy. Compute the mean CV error and its standard error $\mathrm{SE} = s/\sqrt{K}$ for each setting. Choose the **simplest** model whose mean error is within one SE of the best. (ESL §7.10; used by `glmnet`'s `lambda.1se`.)

Step 4 — Baselines. Every reported score needs a trivial model to compare against:

- classification: predict the **majority class** (accuracy = 1 – prevalence: 0.90 at 10% positives, the accuracy paradox of L01 slide 16), or predict the prior probability (Brier score $\pi(1-\pi)$);
- regression: predict the **training mean** (this baseline defines $R^2 = 0$, A6);
- time series: predict **yesterday's value** (persistence).

Step 5 — Ablations and controls. An **ablation** removes one component (a feature group, a preprocessing step, a model part) and re-measures. If the score does not drop, that component was not doing the work you think. A **control** is a run where the answer is known: shuffle the labels and retrain. The score must fall to the baseline; if it doesn't, something is leaking.

Step 6 — Learning curves. Plot training and validation error against training-set size n:

- **both high and close together** ⇒ high bias: more data will not help; use a richer model or better features;
- **large gap, validation still falling** ⇒ high variance: more data, regularisation (L07), or bagging (L12b) will help.

A5 Optimization: learning rate, convergence, SGD

Step 1 — The update (L02 A6): $\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$.

Step 2 — Exact analysis on a quadratic. Near its minimum every smooth loss looks like $J(\theta) \approx J^* + \frac{1}{2}(\theta - \theta^*)^\top H (\theta - \theta^*)$ (L06 A4). In the eigenbasis of H (eigenvalues μ_j), the error along direction j is multiplied by the same factor at every step:

$$e_{j,t+1} = (1 - \eta\mu_j)\, e_{j,t}.$$

This is the $1 - \eta\mu$ factor of L08 A5.

Step 3 — When it converges. We need $|1 - \eta\mu_j| < 1$ for **every** j:

$$\boxed{0 < \eta < \frac{2}{\mu_{\max}}}$$

The **largest** curvature caps the step size; the **smallest** curvature sets the speed. Their ratio is the **condition number** $\kappa = \mu_{\max}/\mu_{\min}$ (L05 A7). The best fixed step is $\eta = 2/(\mu_{\max} + \mu_{\min})$, with factor $(\kappa - 1)/(\kappa + 1)$ per step.

Step 4 — Worked example. $\mu_{\max} = 10$, $\mu_{\min} = 0.1$, so $\kappa = 100$.

η	factor, steep direction $1 - 10\eta$	factor, flat direction $1 - 0.1\eta$	behaviour
0.05	0.5	0.995	smooth but very slow
0.15	−0.5	0.985	steep direction zig-zags but shrinks
0.19	−0.9	0.981	near the limit; zig-zags hard
0.21	**−1.1**	0.979	**diverges**: steep error grows 10% per step

Even at η = 0.19, shrinking the flat-direction error to 1% takes $\ln 0.01 / \ln 0.981 \approx 240$ steps; at η = 0.05 it takes ≈ 920. This is the "crawl" of L08 slide 32.

Step 5 — Two cures for a large κ.

- **Standardise the features** (L07 A10). Unequal feature scales make $X^\top X$ badly conditioned even without collinearity.
- **Use curvature**: Newton/IRLS (L06 A12) divides each direction by its own μ_j, so every factor becomes 0 on a quadratic. Ridge's $+\lambda$ also lowers κ (L07 A5).

Step 6 — Stochastic and mini-batch gradient descent. The full gradient $\nabla J = \frac{1}{n}\sum_i \nabla \ell_i$ costs n evaluations. **SGD** uses one random row; **mini-batch** uses b random rows:

$$g_t = \frac{1}{b}\sum_{i \in \mathcal{B}_t} \nabla \ell_i(\theta_t), \qquad \mathbb{E}[g_t] = \nabla J(\theta_t), \qquad \mathrm{Var}(g_t) \propto \frac{1}{b}.$$

The step is unbiased but noisy. With a fixed η the iterates **bounce** around the minimum instead of settling, so η is decreased over time (a schedule such as $\eta_t \propto 1/t$). Pegasos for SVMs (L11 A15) is exactly

this. **Momentum** keeps a running average of past gradients: it damps the zig-zag across the valley and speeds travel along it.

Step 7 — Convergence diagnostics. Plot the training loss against iterations. If it rises or oscillates, η is too large. If it falls slowly in a straight line on a log scale, the problem is ill-conditioned: rescale. Stop when the relative change $|J_{t+1} - J_t|/|J_t|$ or $\|\nabla J\|$ is small, or by **early stopping** on validation loss (L12c A11).

A6 Regression in practice: R-squared and residual diagnostics

Step 1 — R². Compare the model's SSE with the mean baseline's:

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

Worked: $y = (2, 4, 5, 7, 9)$, $\hat{y} = (2.5, 3.5, 5.5, 7, 8.5)$. SSE = 0.25 + 0.25 + 0.25 + 0 + 0.25 = 1.0; $\bar{y} = 5.4$, SST = 11.56 + 1.96 + 0.16 + 2.56 + 12.96 = 29.2. $R^2 = 1 - 1/29.2 = 0.966$.

Facts. $R^2 = 0$ means no better than the mean; on **test** data R^2 can be **negative**. Training R^2 never falls when you add a feature (the column space only grows, L04 A8), so compare models with validation R^2 or RMSE, not training R^2.

Step 2 — Residual plots. Plot $r_i = y_i - \hat{y}_i$ against $\hat{y}_i$ (and against each feature):

pattern	meaning	try
random band around 0	assumptions look fine	—
curve (U-shape)	missing non-linearity	add x^2, interactions, or use a tree
funnel (spread grows with $\hat{y}$)	non-constant noise variance	model $\log y$; weighted least squares
a few huge residuals	outliers / heavy tails	check the data; absolute or Huber loss (L04 A5)
pattern over time / row order	correlated errors	time-aware model and time-series split (A4)

Least squares is the Gaussian MLE (L04 slide 16). The residual plot is how you check that the Gaussian assumption holds.

A7 Multiclass classification: softmax, one-vs-rest, macro/micro

Step 1 — From one score to K scores. Logistic regression (L06) has one score z and $p = \sigma(z)$. With K classes, give each class a score $z_k = \theta_k^\top x$ and use the **softmax**:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}.$$

All $p_k > 0$ and they sum to 1. Adding the same constant to every z_k changes nothing, so one class can be fixed at $z = 0$. With $K = 2$ this gives back the sigmoid: $p_1 = \frac{e^{z_1}}{e^{z_0}+e^{z_1}} = \sigma(z_1 - z_0)$.

Step 2 — Loss: cross-entropy (negative log-likelihood of a categorical distribution): $\ell = -\ln p_y$.

> **Worked:** $z = (2, 1, 0) \Rightarrow e^z = (7.389, 2.718, 1)$, sum 11.107 $\Rightarrow p = (0.665, 0.245, 0.090)$. If the true class is 1: $\ell = -\ln 0.665 = 0.408$. If it is 3: $\ell = -\ln 0.090 = 2.408$.

Step 3 — Gradient: the same "residual" as before. With one-hot target y,

$$\frac{\partial \ell}{\partial z_k} = p_k - y_k \quad \Rightarrow \quad \nabla_{\theta_k} \ell = (p_k - y_k)\, x,$$

which is exactly $(p - y)x$ from L06 slide 30, one copy per class. The loss is still convex in θ.

Step 4 — Binary-only models. SVMs (L11) and AdaBoost (L12d) are binary. Two standard wrappers:

- **One-vs-rest (OvR):** K classifiers ("class k or not"); predict the class with the highest score.
- **One-vs-one (OvO):** $K(K-1)/2$ classifiers, one per pair; majority vote (scikit-learn's `SVC` does this).

Trees, random forests and KNN are naturally multiclass: a leaf or a neighbourhood holds class fractions.

Step 5 — Metrics with K classes. Compute precision/recall/F1 **per class** (that class vs the rest, L09), then combine:

- **macro** average: plain mean over classes, so rare classes count equally;
- **weighted** average: weighted by class support;

- **micro** average: pool all TP/FP/FN first. For single-label multiclass, micro-F1 = accuracy.

If a rare class matters, report macro-F1 or the per-class table; accuracy and micro-F1 hide a failing rare class (the accuracy paradox again).

A8 Trees II: regression splits, categories, missing values, importance

Step 1 — Regression trees: impurity = SSE. For a numeric target, a leaf predicts the **mean** of its y values, and a split is scored by the drop in squared error:

$$\text{gain} = \text{SSE}(\text{parent}) - \text{SSE}(\text{left}) - \text{SSE}(\text{right}), \qquad \text{SSE}(S) = \sum_{i \in S} (y_i - \bar{y}_S)^2.$$

(This is the "variance reduction" criterion. L12c's $(\sum r)^2/n$ score, L12c A4, is the same quantity written for residuals.)

Worked: $x = 1..6$, $y = (1, 1.2, 0.9, 5, 5.2, 4.8)$; SST = 23.728.

threshold	1.5	2.5	3.5	4.5	5.5
SSE(left) + SSE(right)	18.848	12.708	**0.127**	11.928	19.912

The best split is $x \le 3.5$, with leaves predicting 1.033 and 5.0 (gain 23.60). A regression tree's prediction is a **step function**: it cannot extrapolate beyond the training range of y.

Step 2 — Categorical features. A feature with m categories allows $2^{m-1} - 1$ yes/no groupings. For binary classification (and for regression) you do not have to search them all: sort the categories by their fraction of class 1 (or by mean y) and try only the $m - 1$ cut points of that order. This is provably optimal (ESL §9.2.4). Libraries differ: scikit-learn's `DecisionTreeClassifier` needs numeric (e.g. one-hot) input; LightGBM, CatBoost and `HistGradientBoosting` handle categories natively.

Step 3 — The high-cardinality trap. Information gain favours features with many values. On the twelve runs, a "run ID" feature with 12 distinct values sends each run to its own pure leaf: gain $= 1 - 0 = 1$, beating the real winner rate ≤ 0.065 (0.4591). It is useless for new runs. **Gain ratio** (C4.5) divides by the split's own entropy:

$$\text{GR} = \frac{\text{gain}}{-\sum_k \frac{n_k}{n} \log_2 \frac{n_k}{n}}: \qquad \text{run ID: } \frac{1}{\log_2 12} = \frac{1}{3.585} = 0.279, \qquad \text{rate} \le 0.065\text{: } \frac{0.4591}{H(\frac{4}{12}) = 0.9183} =$$

Gain ratio puts rate first again.

Step 4 — Missing values. Three strategies:

- **Impute** (median, or a "missing" category), fitted inside the training fold (A2);
- **Surrogate splits** (CART/ `rpart`): at each node, also store the next-best question that sends runs the same way; use it when the main feature is missing;
- **Learned default direction** (XGBoost, LightGBM, scikit-learn ≥ 1.3 trees): try sending all missing rows left, then right, and keep the side with the higher gain.

Step 5 — Feature importance.

Impurity-based (MDI): add up, for each feature, (fraction of rows reaching the node) × gain over all its splits. On the twelve-run tree (L12a A5–A7):

feature	node	$\frac{n_{\text{node}}}{n} \times$ gain	importance
rate	root, 12 runs	1×0.4591	**0.459**
batch	runs 5–12	$\frac{8}{12} \times 0.4669$	**0.311**
warm-up	runs 5, 6, 10	$\frac{3}{12} \times 0.9183$	**0.230**

They sum to 1.000 = the root entropy, because every leaf is pure. In a forest, average this over trees.

Permutation importance: shuffle one feature's column in the **validation** data and measure how much the score drops. It is model-agnostic and measured on held-out data. MDI is computed on training data and is biased toward high-cardinality features (Step 3); with **correlated** features both methods split or hide the credit. Importance measures what the model *uses*, not what *causes* the outcome.

Step 6 — Practical properties of trees. No feature scaling needed (a question compares one feature to a threshold, so any monotone rescaling gives the same tree). Robust to outliers in x. Handles interactions automatically. Weaknesses: axis-aligned cuts approximate a diagonal boundary with a staircase; no extrapolation; unstable (L12a A10).

A9 K-nearest neighbours

Step 1 — The algorithm. Store the training set; that is all the "training" there is. To predict at x_0: find the k training points closest to x_0; for classification, return the class fractions among them (majority vote); for regression, return their average y.

Step 2 — k is the bias–variance dial.

- $k = 1$: training error 0 (each point is its own neighbour); jagged boundary; high variance. In 1-D it behaves exactly like the fully grown tree of L12b A8, predicting the y of the nearest point.
- $k = n$: predicts the overall majority / mean everywhere; high bias.
- Choose k by CV (A4), never by training error. For two classes, an odd k avoids ties.

Step 3 — Distance needs comparable scales. Worked on the twelve runs. New run **B** (rate 0.2, batch 64). Use Euclidean distance on (rate, batch), $k = 3$.

Raw units. Batch differs by 32 or more between runs, while rate never differs by more than 0.3. So batch decides everything: only the batch-64 runs can be near B.

neighbour	rate, batch	distance	class
run 4	0.1, 64	0.100	D
run 8	0.03, 64	0.170	C
run 11	0.001, 64	0.199	C

Vote 1 D : 2 C ⇒ **C**.

Log scale, then standardised ($u = \log_{10}\text{rate}$, $v = \log_2 \text{batch}$, each standardised with the training mean and sd: $\bar{u} = -1.635$, $s_u = 0.788$; $\bar{v} = 6.333$, $s_v = 1.179$). B becomes $(1.188, -0.283)$.

neighbour	distance	class
run 4	0.382	D
run 2	0.877	D
run 3	0.931	D

Vote 3 D : 0 C ⇒ **D**, agreeing with the tree (L12a A8), which uses rate as its first question. **Changing the units changed the answer.** KNN is not scale-invariant (trees are, A8 Step 6), so scale the features as in L08 slides 49–51, with training statistics only.

Step 4 — Variants and costs.

- **Distance-weighted** vote: weight each neighbour by $1/d$, so closer neighbours count more.

- **Metric:** Euclidean (ℓ_2), Manhattan (ℓ_1), cosine (for text/embeddings). Categorical features need a different distance (e.g. Hamming).
- **Cost:** no training, but each prediction scans all n points ($O(nd)$). k-d trees and ball trees speed this up in low dimensions; in high dimensions they fail (A10).
- **Theory:** as $n \to \infty$, 1-NN's error is at most **twice** the Bayes error (Cover & Hart, 1967), with no model assumed at all.

A10 The curse of dimensionality

Step 1 — Neighbourhoods stop being local. Spread points uniformly in the unit cube $[0,1]^d$. To capture a fraction r of the data, a sub-cube needs edge length $r^{1/d}$. For $r = 10\%$:

d	1	2	3	10	100
edge $0.1^{1/d}$	0.10	0.32	0.46	**0.79**	**0.98**

In 10-D, the "10 nearest percent" spans 79% of every feature's range, so it is not a neighbourhood any more (ESL §2.5).

Step 2 — Distances concentrate. With many independent noisy features, the nearest and farthest neighbours end up at almost the same distance (the ratio → 1). "Nearest" then carries little information.

Step 3 — Data needed grows exponentially. To keep the same spacing (say 10 points per axis) you need 10^d points.

Step 4 — Why other models survive. Linear models assume structure (a hyperplane), and their error grows only about linearly in d. Trees and lasso pick out the few relevant features. PCA (L05) finds the few directions that carry the spread. Real data usually lie near a low-dimensional surface, which is why KNN on good **embeddings** still works.

Part C Practice

1. **Leakage hunt.** For each, name the kind of leakage (A2) and the fix: (a) predicting loan default with the feature "number of collection calls"; (b) imputing missing ages with the mean of the whole dataset before CV; (c) 5 X-rays per patient, split by image; (d) random 80/20 split of daily sales to forecast next month.
2. **Prior shift.** A test has recall 0.9 and FPR 0.05. Find its precision at prevalence 20% and at 1%. Does its AUC change?
3. **Which CV?** (a) 1 000 emails, 3% spam; (b) sensor readings every minute for a year; (c) 40 hospitals, many patients each, deploy to new hospitals; (d) you tune 3 hyperparameters and must report an honest score.
4. **Step size.** The Hessian of a quadratic loss has eigenvalues 4 and 0.5. (a) The largest η that converges? (b) κ? (c) The best fixed η and its per-step factor?
5. **SGD.** Why does SGD with a fixed η not converge exactly to the minimum, while full gradient descent does? What does a mini-batch of size b do to the gradient noise?
6. **R².** Why can test R^2 be negative, but training R^2 of OLS with an intercept cannot?
7. **Softmax.** Scores $z = (1, 1, 1)$, true class 1. Find p, the loss, and $\partial \ell / \partial z$.
8. **Macro vs weighted.** Per-class F1 is 0.9, 0.8, 0.1 with supports 80, 15, 5. Macro-F1? Weighted-F1? Which one exposes the problem?
9. **Regression split.** $x = (1, 2, 3, 4)$, $y = (2, 2, 6, 8)$. Find SST, the best threshold, its SSE, and the leaf values.
0. **Gini importance.** With Gini, the twelve-run tree has the same three questions, with gains 0.250 (root), 0.2083 (runs 5–12) and 0.4444 (runs 5, 6, 10). Compute the normalised MDI importances.
1. **KNN.** (a) Why is the training error of 1-NN always 0, and what should you use instead? (b) In the A9 example, raw-unit 3-NN for run **A** (0.05, 32) picks runs 5, 6, 10. What does it predict, and which feature decided the neighbours?
2. **Curse.** Edge length of a cube holding 1% of uniform data in $d = 5$ and $d = 50$?
3. **Trees vs KNN.** You multiply the batch feature by 1 000. What happens to (a) the tree of L12a, (b) the raw-unit KNN prediction?

1. (a) Target leakage: calls happen *after* the default starts. Drop the feature. (b) Preprocessing leakage: impute inside each training fold. (c) Group leakage: split by patient (group K-fold). (d) Temporal leakage: use a time split (train on earlier days, test on later ones).

2. 20%: $\frac{0.18}{0.18+0.04} = 0.818$. 1%: $\frac{0.009}{0.009+0.0495} = 0.154$. AUC does not change: TPR and FPR are both computed within one class.
3. (a) Stratified K-fold (a random fold could hold almost no spam). (b) Time-series split. (c) Group K-fold by hospital. (d) Nested CV.
4. (a) η < 2/4 = 0.5. (b) κ = 4/0.5 = 8. (c) η = 2/(4 + 0.5) = 0.444; factor (8 – 1)/(8 + 1) = 0.778 in both directions (1 – 0.444·4 = –0.778, 1 – 0.444·0.5 = 0.778).
5. At the minimum the **full** gradient is 0, but a single row's gradient is not, so each SGD step still moves. The iterates jitter in a region of size $\propto$ η, which is why η is decreased over time. A mini-batch divides the gradient variance by b.
6. On test data the model can be worse than the training mean, so SSE > SST. With an intercept, OLS can always reproduce the mean (θ = ($\bar{y}$, 0, …)), and it minimises training SSE, so SSE ≤ SST.
7. $p = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$; $\ell = \ln 3 = 1.099$; $\partial \ell / \partial z = p - y = (-\frac{2}{3}, \frac{1}{3}, \frac{1}{3})$.
8. Macro: (0.9 + 0.8 + 0.1)/3 = 0.600. Weighted: 0.8·0.9 + 0.15·0.8 + 0.05·0.1 = 0.720 + 0.120 + 0.005 = 0.845. Macro exposes the failing rare class.
9. $\bar{y} = 4.5$, SST = 6.25 + 6.25 + 2.25 + 12.25 = 27. Candidates: 1.5 → 0 + 18.667; **2.5 → 0 + 2 = 2**; 3.5 → 10.667 + 0. Best $x \leq 2.5$, SSE 2 (gain 25), leaves 2 and 7.
10. Raw: rate 1·0.250 = 0.250; batch $\frac{8}{12}$·0.2083 = 0.1389; warm-up $\frac{3}{12}$·0.4444 = 0.1111. Total 0.5 (the root Gini). Normalised: 0.500, 0.278, 0.222.
11. (a) Each training point is its own nearest neighbour at distance 0. Use CV, e.g. leave-one-out: the nearest **other** point. (b) Runs 5, 6 are D and 10 is C ⇒ **D**. Batch decided: only batch-32 runs are within 32 units.
12. $0.01^{1/5} = 0.398$; $0.01^{1/50} = 0.912$.
13. (a) Nothing: the threshold becomes 48 000 and the same runs go each way. (b) The rate feature becomes even more irrelevant to the distance. The neighbours are still the same-batch runs, so here the vote does not change, but any rate information is drowned out completely.

Back to the start of Lecture 12: L12a — Decision trees.